

Step-by-Step Java & Spring Boot Environment Setup

This guide will take you from having zero developer tools installed to running a blank Spring Boot application. Follow these steps in order.

Step 1: Install the Java Development Kit (JDK)

To run and write Spring Boot applications, you need the **Java Development Kit (JDK)**. We will use **Java 17** or **Java 21** (LTS - Long Term Support versions).

1. Download the JDK

We recommend **Eclipse Temurin** by Adoptium (a highly reliable, free, open-source Java distribution).

1. Go to: <https://adoptium.net/temurin/releases/>
2. Select your Operating System (Windows, macOS, or Linux).
3. Select **Architecture** (for Windows: `x64` ; for modern Macs: `aarch64` / Apple Silicon; for older Intel Macs: `x64`).
4. Download the **Installer** (e.g., `.msi` for Windows, `.pkg` for Mac).
5. Run the downloaded installer and accept all default configurations. **On Windows, make sure to check the option that says "Set JAVA_HOME variable" and "Associate .jar files" during installation.**

2. Verify Your Installation

Once the installer completes, open your system's Command Prompt (Windows) or Terminal (Mac) and type:

```
java -version
```

You should see output similar to this:

```
openjdk version "21.0.x" ...
OpenJDK Runtime Environment ...
OpenJDK 64-Bit Server VM ...
```

Next, check the Java Compiler:

```
javac -version
```

If both commands return a version number, your Java environment is successfully installed!

Step 2: Configure Environment Variables (Mainly for Windows)

Sometimes your system doesn't automatically know where Java is located. We need to set up the `JAVA_HOME` variable.

For Windows:

1. Search for "**Environment Variables**" in your Windows search bar and click "**Edit the system environment variables**".
2. Click the "**Environment Variables...**" button at the bottom right.
3. Under **System Variables**, click **New...**:
 - **Variable name:** `JAVA_HOME`
 - **Variable value:** The path where Java was installed (usually `C:\Program Files\Eclipse Adoptium\jdk-21.x.x...` or similar).
4. Find the `Path` variable under System Variables, select it, and click **Edit...**
5. Click **New** and add: `%JAVA_HOME%\bin`
6. Click **OK** on all open windows to save.
7. Restart your terminal/command prompt to apply.

Step 3: Understand the Project Build Tool (Maven)

Spring Boot applications use a build tool to manage external libraries (like database connectors, web utilities, etc.) and compile your code. The most common tool is **Maven**.

The best part: You do **not** need to install Maven manually on your computer! When we generate a Spring Boot project, it includes a small file called `mvnw` (Maven Wrapper). This wrapper will automatically download and install the exact version of Maven needed for our project the first time we run it.

Step 4: Bootstrap your Spring Boot Project

We use the official utility called **Spring Initializr** to generate our skeleton project.

1. Open your web browser and go to <https://start.spring.io>.
2. Configure the project options precisely as follows:
 - **Project:** `Maven`
 - **Language:** `Java`
 - **Spring Boot:** Select the latest stable version (e.g., `3.2.x` or `3.3.x` — avoid versions marked with `SNAPSHOT` or `M`).
 - **Project Metadata:**
 - **Group:** `com.hisrah`

- **Artifact:** `accounting`
- **Name:** `accounting`
- **Description:** HISRAH Accounting Module API
- **Package name:** `com.hisrah.accounting`
- **Packaging:** `Jar`
- **Java:** `17` or `21` (Choose the version matching what you installed in Step 1).

3. **Add Dependencies** (Click the "Add Dependencies" button on the right):

- Select **Spring Web** (Enables building REST APIs).
- Select **Spring Data JPA** (Handles database connections and tables).
- Select **H2 Database** (An in-memory SQL database that requires zero configuration).
- Select **Validation** (Enables checking input data formats like emails or numeric constraints).
- Select **Lombok** (Reduces repetitive Java code like getters and setters).

4. Click the large **GENERATE** button at the bottom.

5. This downloads a file named `accounting.zip` to your computer.

Step 5: Unzip and Open the Project

1. Extract (unzip) `accounting.zip` to a folder on your computer where you want to keep your projects (e.g., `C:\Projects\accounting` or `~/Projects/accounting`).
2. Open your preferred code editor (such as Cursor or VS Code).
3. Go to **File > Open Folder** (or **Open...**) and select the extracted `accounting` directory.
4. Let the editor load. You will see a file layout like this:

```

accounting/
├── .mvn/                # Maven Wrapper configurations
├── src/
│   ├── main/
│   │   ├── java/        # Your Java source code files
│   │   └── resources/    # Configuration files and database properties
│   └── test/            # Your Unit and Integration tests
├── mvnw                 # Maven Wrapper script for Linux/macOS
├── mvnw.cmd             # Maven Wrapper script for Windows
└── pom.xml              # Maven configuration where dependencies are managed

```

Step 6: Start Your Spring Boot Application!

Now we are going to make sure the app starts on your computer.

1. Open the built-in terminal in your code editor.
2. Run the startup command based on your Operating System:

- **On Windows (Command Prompt / Powershell):**

```
mvnw.cmd spring-boot:run
```

- **On macOS / Linux:**

```
./mvnw spring-boot:run
```

3. What happens next:

- Maven will download all required libraries from the internet. This might take 1–3 minutes on your first run. Please be patient!
- You will see some console logs scroll past. Look for a line that resembles this at the end:

```
[INFO] Started AccountingApplication in 2.345 seconds (process running fo
```

- It is officially running! By default, Spring Boot starts a web server on port 8080 .

4. Verify it works:

- Open your browser and go to `http://localhost:8080` .
- You should see a generic white page displaying a `Whitelabel Error Page` (this is expected and perfect! It means the server is online and responding, but we haven't created any homepage yet).

— — — — —